

Project Report - CLG MANAGEMENT

Project Description

CLG MANAGEMENT is a full-stack college management system built to manage core academic records in one place. It provides user authentication and allows admins/users to create, update, view, and delete students, courses, and enrollments. The project uses a React frontend for the user interface and a Node.js + Express + MongoDB backend for API and data persistence.

Project Information

- **Project Name:** CLG MANAGEMENT
- **Project Type:** Full-stack web application
- **Frontend:** React (Vite), React Router, Tailwind CSS
- **Backend:** Node.js, Express.js
- **Database:** MongoDB (Mongoose)
- **Core Modules:** Authentication, Students, Courses, Enrollments, Dashboard
- **Default API Base URL:** `http://localhost:3000/api`

Codebase Structure

- `backend/` - Server, routes, models, API logic
- `frontend/src/` - UI components, pages, routing, API client

Folder Structure

```
CLG MANAGEMENT/
├── backend/
│   ├── .env
│   ├── docker-compose.yml
│   ├── index.js
│   ├── model.js
│   ├── route.js
│   └── package.json
├── frontend/
│   ├── package.json
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── App.css
│       ├── api.js
│       ├── index.css
│       ├── components/
│       │   ├── AppLayout.jsx
│       │   └── ProtectedRoute.jsx
│       └── pages/
│           ├── LoginPage.jsx
│           ├── RegisterPage.jsx
│           ├── DashboardPage.jsx
│           └── StudentsPage.jsx
```

```
|           |— CoursesPage.jsx
|           |— EnrollmentsPage.jsx
|— project-report.md
```

Project Objectives

1. Provide a centralized platform to manage students, courses, and enrollments.
2. Offer a simple authentication flow (register/login) for secure access.
3. Enable full CRUD operations for core academic records.
4. Show a dashboard with live counts for quick administrative overview.

Functional Workflow

1. User registers and logs in.
2. User adds students and courses.
3. User creates enrollments by mapping student + course + semester.
4. Dashboard and listing pages display and manage all saved records.

Backend API Summary

- **Auth:** `/api/auth/register` , `/api/auth/login`
- **Students:** create, list, get by id, update, delete, next unique id
- **Courses:** create, list, get by id, update, delete
- **Enrollments:** create and list with populated student/course details

Complete Project Code

backend/package.json

Work of this code: Defines backend package metadata, startup script, and dependencies required to run the Node.js API server.

```
{
  "name": "clg-management-backend",
  "version": "1.0.0",
  "private": true,
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "mongoose": "^9.6.1"
  }
}
```

backend/index.js

Work of this code: Starts the Express server, configures middleware (CORS + JSON), connects MongoDB, mounts API routes, and handles common server errors.

```

require("dotenv").config();
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
const apiRoutes = require("./route");

const app = express();
const PORT = process.env.PORT || 3000;
const MONGODB_URI =
  process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/clg_management";

app.use(cors({ origin: process.env.CORS_ORIGIN || "http://localhost:5173" }));
app.use(express.json());

app.get("/", (_req, res) => {
  res.status(200).json({ message: "backend running" });
});

app.use("/api", apiRoutes);

app.use((err, _req, res, _next) => {
  if (err?.code === 11000) {
    return res.status(409).json({ message: "duplicate value already exists" });
  }
  return res.status(500).json({ message: err.message || "internal server error" });
});

async function startServer() {
  await mongoose.connect(MONGODB_URI);
  app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
  });
}

startServer();

```

backend/model.js

Work of this code: Declares MongoDB schemas/models for `User` , `Student` , `Course` , and `Enrollment` including validations and unique constraints.

```

const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    email: {
      type: String,
      required: true,
      unique: true,
      trim: true,
      lowercase: true,

```

```
    },
    password: {
      type: String,
      required: true,
    },
  },
  {
    timestamps: true,
  }
);
```

```
const studentSchema = new mongoose.Schema(
  {
    studentUniqueId: {
      type: String,
      required: true,
      unique: true,
      sparse: true,
      immutable: true,
      match: /^\d{12}$/ ,
    },
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      trim: true,
      lowercase: true,
    },
    department: {
      type: String,
      required: true,
      trim: true,
    },
    semester: {
      type: Number,
      required: true,
      min: 1,
      max: 8,
    },
  },
  {
    timestamps: true,
  }
);
```

```
const courseSchema = new mongoose.Schema(
  {
```

```

    code: {
      type: String,
      required: true,
      unique: true,
      trim: true,
      uppercase: true,
    },
    title: {
      type: String,
      required: true,
      trim: true,
    },
    department: {
      type: String,
      required: true,
      trim: true,
    },
    credits: {
      type: Number,
      required: true,
      min: 1,
      max: 10,
    },
  },
},
{
  timestamps: true,
}
);

const enrollmentSchema = new mongoose.Schema(
  {
    enrollmentNumber: {
      type: String,
      required: true,
      unique: true,
      sparse: true,
      match: /^\\d{12}$/,
    },
    student: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Student",
      required: true,
    },
    course: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Course",
      required: true,
    },
    semester: {
      type: Number,
      required: true,
      min: 1,

```

```

        max: 8,
      },
    },
    {
      timestamps: true,
    }
  );

enrollmentSchema.index({ student: 1, course: 1, semester: 1 }, { unique: true });

const User = mongoose.model("User", userSchema);
const Student = mongoose.model("Student", studentSchema);
const Course = mongoose.model("Course", courseSchema);
const Enrollment = mongoose.model("Enrollment", enrollmentSchema);

module.exports = {
  User,
  Student,
  Course,
  Enrollment,
};

```

backend/route.js

Work of this code: Implements all REST API endpoints for authentication, student/course CRUD, and enrollment creation/listing with unique ID/number generation logic.

```

const express = require("express");
const { randomInt } = require("crypto");
const { User, Student, Course, Enrollment } = require("../model");

const router = express.Router();
const STUDENT_UNIQUE_ID_MIN = 100000000000;
const STUDENT_UNIQUE_ID_MAX = 10000000000000;
const STUDENT_UNIQUE_ID_MAX_ATTEMPTS = 10;
const ENROLLMENT_NUMBER_MIN = 100000000000;
const ENROLLMENT_NUMBER_MAX = 10000000000000;
const ENROLLMENT_NUMBER_MAX_ATTEMPTS = 10;

function generateStudentUniqueId() {
  return String(randomInt(STUDENT_UNIQUE_ID_MIN, STUDENT_UNIQUE_ID_MAX));
}

function generateEnrollmentNumber() {
  return String(randomInt(ENROLLMENT_NUMBER_MIN, ENROLLMENT_NUMBER_MAX));
}

router.post("/auth/register", async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {

```

```

    return res.status(400).json({ message: "email and password are required" });
  }

  const existingUser = await User.findOne({ email: email.toLowerCase() });
  if (existingUser) {
    return res.status(409).json({ message: "email already exists" });
  }

  await User.create({ email, password });

  return res.status(201).json({ message: "user registered successfully" });
});

router.post("/auth/login", async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: "email and password are required" });
  }

  const user = await User.findOne({ email: email.toLowerCase() });
  if (!user) {
    return res.status(401).json({ message: "invalid credentials" });
  }

  if (user.password !== password) {
    return res.status(401).json({ message: "invalid credentials" });
  }

  return res.status(200).json({ message: "login success", email: user.email });
});

router.post("/students", async (req, res) => {
  const { studentUniqueId: requestedStudentUniqueId, ...studentPayload } = req.body;
  let student = null;

  for (let attempt = 0; attempt < STUDENT_UNIQUE_ID_MAX_ATTEMPTS; attempt += 1) {
    const studentUniqueId =
      attempt === 0 && requestedStudentUniqueId
        ? requestedStudentUniqueId
        : generateStudentUniqueId();
    try {
      student = await Student.create({
        ...studentPayload,
        studentUniqueId,
      });
      break;
    } catch (error) {
      if (error?.code === 11000 && error?.keyPattern?.studentUniqueId) {
        continue;
      }
      throw error;
    }
  }
});

```

```

    }
  }

  if (!student) {
    return res.status(500).json({ message: "failed to generate unique student id"
});
  }

  return res.status(201).json(student);
});

router.get("/students", async (_req, res) => {
  const students = await Student.find().sort({ createdAt: -1 });
  return res.status(200).json(students);
});

router.get("/students/next-unique-id", async (_req, res) => {
  for (let attempt = 0; attempt < STUDENT_UNIQUE_ID_MAX_ATTEMPTS; attempt += 1) {
    const studentUniqueId = generateStudentUniqueId();
    const existingStudent = await Student.findOne({ studentUniqueId
}).select("_id");
    if (!existingStudent) {
      return res.status(200).json({ studentUniqueId });
    }
  }

  return res.status(500).json({ message: "failed to generate unique student id" });
});

router.get("/students/:id", async (req, res) => {
  const student = await Student.findById(req.params.id);
  if (!student) {
    return res.status(404).json({ message: "student not found" });
  }
  return res.status(200).json(student);
});

router.put("/students/:id", async (req, res) => {
  const { studentUniqueId, ...updatablePayload } = req.body;
  const student = await Student.findByIdAndUpdate(req.params.id, updatablePayload, {
    new: true,
    runValidators: true,
  });
  if (!student) {
    return res.status(404).json({ message: "student not found" });
  }
  return res.status(200).json(student);
});

router.delete("/students/:id", async (req, res) => {
  const deleted = await Student.findByIdAndDelete(req.params.id);
  if (!deleted) {

```

```

        return res.status(404).json({ message: "student not found" });
    }
    return res.status(200).json({ message: "student deleted" });
});

router.post("/courses", async (req, res) => {
    const course = await Course.create(req.body);
    return res.status(201).json(course);
});

router.get("/courses", async (_req, res) => {
    const courses = await Course.find().sort({ createdAt: -1 });
    return res.status(200).json(courses);
});

router.get("/courses/:id", async (req, res) => {
    const course = await Course.findById(req.params.id);
    if (!course) {
        return res.status(404).json({ message: "course not found" });
    }
    return res.status(200).json(course);
});

router.put("/courses/:id", async (req, res) => {
    const course = await Course.findByIdAndUpdate(req.params.id, req.body, {
        new: true,
        runValidators: true,
    });
    if (!course) {
        return res.status(404).json({ message: "course not found" });
    }
    return res.status(200).json(course);
});

router.delete("/courses/:id", async (req, res) => {
    const deleted = await Course.findByIdAndDelete(req.params.id);
    if (!deleted) {
        return res.status(404).json({ message: "course not found" });
    }
    return res.status(200).json({ message: "course deleted" });
});

router.post("/enrollments", async (req, res) => {
    const { studentId, courseId, semester } = req.body;
    if (!studentId || !courseId || !semester) {
        return res
            .status(400)
            .json({ message: "studentId, courseId and semester are required" });
    }

    const [student, course] = await Promise.all([
        Student.findById(studentId),

```

```

    Course.findById(courseId),
  });

  if (!student) {
    return res.status(404).json({ message: "student not found" });
  }
  if (!course) {
    return res.status(404).json({ message: "course not found" });
  }

  let enrollment = null;
  for (let attempt = 0; attempt < ENROLLMENT_NUMBER_MAX_ATTEMPTS; attempt += 1) {
    try {
      enrollment = await Enrollment.create({
        enrollmentNumber: generateEnrollmentNumber(),
        student: studentId,
        course: courseId,
        semester,
      });
      break;
    } catch (error) {
      if (error?.code === 11000 && error?.keyPattern?.enrollmentNumber) {
        continue;
      }
      throw error;
    }
  }

  if (!enrollment) {
    return res.status(500).json({ message: "failed to generate unique enrollment number" });
  }

  const populatedEnrollment = await Enrollment.findById(enrollment._id)
    .populate("student", "studentUniqueId name email department semester")
    .populate("course", "code title department credits");

  return res.status(201).json(populatedEnrollment);
});

router.get("/enrollments", async (_req, res) => {
  const enrollments = await Enrollment.find()
    .sort({ createdAt: -1 })
    .populate("student", "studentUniqueId name email department semester")
    .populate("course", "code title department credits");
  return res.status(200).json(enrollments);
});

module.exports = router;

```

frontend/package.json

Work of this code: Defines frontend scripts and dependencies for the React + Vite application.

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.5",
    "react-dom": "^19.2.5",
    "react-router-dom": "^7.14.2"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "autoprefixer": "^10.5.0",
    "eslint": "^10.2.1",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.5.0",
    "postcss": "^8.5.13",
    "tailwindcss": "^3.4.17",
    "vite": "^8.0.10"
  }
}
```

frontend/src/main.jsx

Work of this code: Entry point of the frontend app; mounts React root and enables routing with `BrowserRouter`.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

```
    </StrictMode>,  
  )
```

frontend/src/App.jsx

Work of this code: Configures all application routes, route protection, and layout wrapping for protected pages.

```
import { Navigate, Route, Routes } from "react-router-dom";  
import AppLayout from "../components/AppLayout";  
import ProtectedRoute from "../components/ProtectedRoute";  
import CoursesPage from "../pages/CoursesPage";  
import DashboardPage from "../pages/DashboardPage";  
import EnrollmentsPage from "../pages/EnrollmentsPage";  
import LoginPage from "../pages/LoginPage";  
import RegisterPage from "../pages/RegisterPage";  
import StudentsPage from "../pages/StudentsPage";  
  
function App() {  
  const currentUser = localStorage.getItem("cms-user-email");  
  
  return (  
    <Routes>  
      <Route  
        path="/"  
        element={<Navigate to={currentUser ? "/dashboard" : "/login"} replace />}  
      />  
      <Route path="/login" element={<LoginPage />} />  
      <Route path="/register" element={<RegisterPage />} />  
      <Route  
        path="/dashboard"  
        element={(  
          <ProtectedRoute>  
            <AppLayout title="Dashboard">  
              <DashboardPage />  
            </AppLayout>  
          </ProtectedRoute>  
        )}  
      />  
      <Route  
        path="/students"  
        element={(  
          <ProtectedRoute>  
            <AppLayout title="Students">  
              <StudentsPage />  
            </AppLayout>  
          </ProtectedRoute>  
        )}  
      />  
      <Route  
        path="/courses"
```

```

        element={
          <ProtectedRoute>
            <AppLayout title="Courses">
              <CoursesPage />
            </AppLayout>
          </ProtectedRoute>
        }
      />
    <Route
      path="/enrollments"
      element={
        <ProtectedRoute>
          <AppLayout title="Enrollments">
            <EnrollmentsPage />
          </AppLayout>
        </ProtectedRoute>
      }
    />
    <Route path="*" element={<Navigate to="/" replace />} />
  </Routes>
);
}

export default App;

```

frontend/src/api.js

Work of this code: Central API utility for sending HTTP requests to backend and handling common error responses.

```

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ||
"http://localhost:3000/api";

export function getApiBaseUrl() {
  return API_BASE_URL;
}

export async function apiRequest(path, options = {}) {
  const response = await fetch(`${API_BASE_URL}${path}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || "request failed");
  }

  return data;
}

```

frontend/src/components/ProtectedRoute.jsx

Work of this code: Prevents unauthenticated users from opening protected routes and redirects them to login.

```
import { Navigate } from "react-router-dom";

function ProtectedRoute({ children }) {
  const currentUser = localStorage.getItem("cms-user-email");

  if (!currentUser) {
    return <Navigate to="/login" replace />;
  }

  return children;
}

export default ProtectedRoute;
```

frontend/src/components/AppLayout.jsx

Work of this code: Provides shared dashboard layout with sidebar navigation, page title area, and logout behavior.

```
import { Link, useLocation, useNavigate } from "react-router-dom";

const links = [
  { to: "/dashboard", label: "Dashboard" },
  { to: "/students", label: "Students" },
  { to: "/courses", label: "Courses" },
  { to: "/enrollments", label: "Enrollments" },
];

function AppLayout({ title, children }) {
  const location = useLocation();
  const navigate = useNavigate();
  const currentUser = localStorage.getItem("cms-user-email");

  function logout() {
    localStorage.removeItem("cms-user-email");
    navigate("/login", { replace: true });
  }

  return (
    <div className="min-h-screen bg-gradient-to-b from-purple-50 to-orange-50">
      <div className="mx-auto grid min-h-screen w-full max-w-7xl gap-4 p-4 md:grid-cols-[260px_1fr] md:p-6">
        <aside className="rounded-2xl bg-white p-4 shadow-sm md:p-5">
          <h1 className="text-lg font-bold text-purple-900">CLG Management</h1>
          <p className="mt-1 text-xs text-slate-500">{currentUser}</p>
          <nav className="mt-6 space-y-2">
```

```

        {links.map((item) => {
          const active = location.pathname === item.to;
          return (
            <Link
              key={item.to}
              to={item.to}
              className={`block rounded-lg px-3 py-2 text-sm font-medium
transition-colors duration-200 ${
                active
                  ? "bg-purple-600 text-white"
                  : "text-slate-700 hover:bg-purple-100 hover:text-purple-900"
              }`}
            >
              {item.label}
            </Link>
          );
        })}
      </nav>
      <button
        type="button"
        onClick={logout}
        className="mt-6 w-full cursor-pointer rounded-lg bg-orange-500 px-3 py-2
text-sm font-medium text-white transition-colors duration-200 hover:bg-orange-600"
      >
        Logout
      </button>
    </aside>

    <main className="rounded-2xl bg-white p-4 shadow-sm md:p-6">
      <header className="mb-6 border-b border-slate-100 pb-4">
        <h2 className="text-2xl font-semibold text-purple-900">{title}</h2>
      </header>
      {children}
    </main>
  </div>
</div>
);
}

export default AppLayout;

```

frontend/src/pages/LoginPage.jsx

Work of this code: Renders login form, calls login API, stores logged-in email in local storage, and navigates to dashboard on success.

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { apiRequest } from "../api";

function LoginPage() {

```

```

const navigate = useNavigate();
const [form, setForm] = useState({ email: "", password: "" });
const [error, setError] = useState("");
const [loading, setLoading] = useState(false);

async function handleSubmit(event) {
  event.preventDefault();
  setLoading(true);
  setError("");

  try {
    const data = await apiRequest("/auth/login", {
      method: "POST",
      body: JSON.stringify(form),
    });
    localStorage.setItem("cms-user-email", data.email);
    navigate("/dashboard", { replace: true });
  } catch (submitError) {
    setError(submitError.message);
  } finally {
    setLoading(false);
  }
}

return (
  <div className="flex min-h-screen items-center justify-center bg-gradient-to-br
from-purple-100 to-orange-100 p-4">
    <div className="w-full max-w-md rounded-2xl bg-white p-6 shadow-lg">
      <h1 className="text-2xl font-bold text-purple-900">Welcome back</h1>
      <p className="mt-1 text-sm text-slate-600">Login to continue managing your
college data.</p>

      <form onSubmit={handleSubmit} className="mt-6 space-y-4">
        <div>
          <label htmlFor="login-email" className="mb-1 block text-sm font-medium
text-slate-700">
            Email
          </label>
          <input
            id="login-email"
            type="email"
            required
            value={form.email}
            onChange={(event) => setForm((prev) => ({ ...prev, email:
event.target.value })))}
            className="w-full rounded-lg border border-slate-200 px-3 py-2
outline-none ring-purple-300 focus:ring-2"
          />
        </div>
        <div>
          <label
            htmlFor="login-password"

```

```

        className="mb-1 block text-sm font-medium text-slate-700"
      >
        Password
      </label>
      <input
        id="login-password"
        type="password"
        required
        value={form.password}
        onChange={(event) =>
          setForm((prev) => ({ ...prev, password: event.target.value }))
        }
        className="w-full rounded-lg border border-slate-200 px-3 py-2
outline-none ring-purple-300 focus:ring-2"
      />
    </div>

    {error ? <p className="rounded bg-rose-50 px-3 py-2 text-sm text-rose-
700">{error}</p> : null}

    <button
      type="submit"
      disabled={loading}
      className="w-full cursor-pointer rounded-lg bg-purple-600 px-4 py-2
font-medium text-white transition-colors duration-200 hover:bg-purple-700
disabled:cursor-not-allowed disabled:bg-purple-300"
    >
      {loading ? "Logging in..." : "Login"}
    </button>
  </form>

  <p className="mt-4 text-sm text-slate-600">
    No account?{" " }
    <Link to="/register" className="font-medium text-purple-700
hover:underline">
      Create one
    </Link>
  </p>
</div>
</div>
);
}

export default LoginPage;

```

frontend/src/pages/RegisterPage.jsx

Work of this code: Renders registration form, creates new user account through API, and redirects user to login after successful registration.

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { apiRequest } from "../api";

function RegisterPage() {
  const navigate = useNavigate();
  const [form, setForm] = useState({ email: "", password: "" });
  const [status, setStatus] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  async function handleSubmit(event) {
    event.preventDefault();
    setLoading(true);
    setError("");
    setStatus("");

    try {
      const data = await apiRequest("/auth/register", {
        method: "POST",
        body: JSON.stringify(form),
      });
      setStatus(data.message);
      setForm({ email: "", password: "" });
      setTimeout(() => navigate("/login", { replace: true }), 700);
    } catch (submitError) {
      setError(submitError.message);
    } finally {
      setLoading(false);
    }
  }

  return (
    <div className="flex min-h-screen items-center justify-center bg-gradient-to-br
from-purple-100 to-orange-100 p-4">
      <div className="w-full max-w-md rounded-2xl bg-white p-6 shadow-lg">
        <h1 className="text-2xl font-bold text-purple-900">Create account</h1>
        <p className="mt-1 text-sm text-slate-600">Get started with CLG Management.
</p>

        <form onSubmit={handleSubmit} className="mt-6 space-y-4">
          <div>
            <label
              htmlFor="register-email"
              className="mb-1 block text-sm font-medium text-slate-700"
            >
              Email
            </label>
            <input
              id="register-email"
              type="email"

```

```

        required
        value={form.email}
        onChange={(event) => setForm((prev) => ({ ...prev, email:
event.target.value })))}
        className="w-full rounded-lg border border-slate-200 px-3 py-2
outline-none ring-purple-300 focus:ring-2"
      />
    </div>
    <div>
      <label
        htmlFor="register-password"
        className="mb-1 block text-sm font-medium text-slate-700"
      >
        Password
      </label>
      <input
        id="register-password"
        type="password"
        required
        value={form.password}
        onChange={(event) =>
          setForm((prev) => ({ ...prev, password: event.target.value }))
        }
        className="w-full rounded-lg border border-slate-200 px-3 py-2
outline-none ring-purple-300 focus:ring-2"
      />
    </div>

    {status ? (
      <p className="rounded bg-emerald-50 px-3 py-2 text-sm text-emerald-700">
{status}</p>
    ) : null}
    {error ? <p className="rounded bg-rose-50 px-3 py-2 text-sm text-rose-
700">{error}</p> : null}

    <button
      type="submit"
      disabled={loading}
      className="w-full cursor-pointer rounded-lg bg-purple-600 px-4 py-2
font-medium text-white transition-colors duration-200 hover:bg-purple-700
disabled:cursor-not-allowed disabled:bg-purple-300"
    >
      {loading ? "Creating..." : "Create account"}
    </button>
  </form>

  <p className="mt-4 text-sm text-slate-600">
    Already have an account?{" "}
    <Link to="/login" className="font-medium text-purple-700 hover:underline">
      Login
    </Link>
  </p>

```

```

        </div>
      </div>
    );
  }

  export default RegisterPage;

```

frontend/src/pages/DashboardPage.jsx

Work of this code: Fetches student/course/enrollment counts and displays a summary dashboard plus workflow guidance.

```

import { useEffect, useState } from "react";
import { getApiBaseUrl, apiRequest } from "../api";

function DashboardPage() {
  const [counts, setCounts] = useState({ students: 0, courses: 0, enrollments: 0 });
  const [error, setError] = useState("");

  useEffect(() => {
    async function load() {
      try {
        const [students, courses, enrollments] = await Promise.all([
          apiRequest("/students"),
          apiRequest("/courses"),
          apiRequest("/enrollments"),
        ]);
        setCounts({
          students: students.length,
          courses: courses.length,
          enrollments: enrollments.length,
        });
      } catch (loadError) {
        setError(loadError.message);
      }
    }

    load();
  }, []);

  const cards = [
    { label: "Total Students", value: counts.students },
    { label: "Total Courses", value: counts.courses },
    { label: "Total Enrollments", value: counts.enrollments },
  ];

  return (
    <div className="space-y-6">
      {error ? <p className="rounded bg-rose-50 px-3 py-2 text-sm text-rose-700">
        {error}</p> : null}
      <div className="grid gap-4 md:grid-cols-3">

```

```

        {cards.map((card) => (
            <div
                key={card.label}
                className="rounded-xl border border-purple-100 bg-purple-50 p-4
transition-colors duration-200 hover:border-purple-300"
            >
                <p className="text-sm text-purple-700">{card.label}</p>
                <p className="mt-2 text-3xl font-semibold text-purple-900">{card.value}
</p>
            </div>
        ))}
    </div>

    <div className="rounded-xl border border-slate-100 bg-slate-50 p-4">
        <h3 className="text-lg font-semibold text-slate-800">System flow</h3>
        <ol className="mt-3 list-decimal space-y-1 pl-5 text-sm text-slate-700">
            <li>Create user account and login.</li>
            <li>Add students and courses.</li>
            <li>Create enrollments by mapping students to courses by semester.</li>
            <li>Use Students, Courses, and Enrollments pages to manage records.</li>
        </ol>
        <p className="mt-4 text-xs text-slate-500">API endpoint: {getApiBaseUrl()}
    </div>
</div>
);
}

export default DashboardPage;

```

frontend/src/pages/StudentsPage.jsx

Work of this code: Manages student records with create/update/delete operations and auto-fetches next available student unique ID.

```

import { useEffect, useState } from "react";
import { apiRequest } from "../api";

const initialForm = {
    id: "",
    studentUniqueId: "",
    name: "",
    email: "",
    department: "",
    semester: 1,
};

function StudentsPage() {
    const [students, setStudents] = useState([]);
    const [form, setForm] = useState(initialForm);
    const [status, setStatus] = useState("");

```

```

const [loading, setLoading] = useState(false);

async function loadStudents() {
  const data = await apiRequest("/students");
  setStudents(data);
}

async function loadNextUniqueId() {
  const data = await apiRequest("/students/next-unique-id");
  setForm((prev) => ({ ...prev, studentUniqueId: data.studentUniqueId }));
}

useEffect(() => {
  let ignore = false;

  async function bootstrap() {
    try {
      const [studentsData, uniqueIdData] = await Promise.all([
        apiRequest("/students"),
        apiRequest("/students/next-unique-id"),
      ]);
      if (!ignore) {
        setStudents(studentsData);
        setForm((prev) => ({ ...prev, studentUniqueId:
uniqueIdData.studentUniqueId }));
      }
    } catch (error) {
      if (!ignore) {
        setStatus(error.message);
      }
    }
  }

  bootstrap();

  return () => {
    ignore = true;
  };
}, []);

async function handleSubmit(event) {
  event.preventDefault();
  setLoading(true);
  setStatus("");
  try {
    const payload = {
      name: form.name,
      email: form.email,
      department: form.department,
      semester: Number(form.semester),
    };
    if (form.id) {

```

```

        await apiRequest(`/students/${form.id}`, {
            method: "PUT",
            body: JSON.stringify(payload),
        });
        setStatus("Student updated successfully.");
    } else {
        await apiRequest("/students", {
            method: "POST",
            body: JSON.stringify({
                ...payload,
                studentUniqueId: form.studentUniqueId,
            }),
        });
        setStatus("Student created successfully.");
    }
    setForm(initialForm);
    await loadStudents();
    await loadNextUniqueId();
} catch (submitError) {
    setStatus(submitError.message);
} finally {
    setLoading(false);
}
}

async function handleDelete(id) {
    try {
        await apiRequest(`/students/${id}`, { method: "DELETE" });
        setStatus("Student deleted successfully.");
        await loadStudents();
    } catch (deleteError) {
        setStatus(deleteError.message);
    }
}

return (
    <div className="space-y-6">
        {status ? <p className="rounded bg-slate-100 px-3 py-2 text-sm text-slate-700">{status}</p> : null}

        <section className="rounded-xl border border-slate-100 p-4">
            <h3 className="mb-4 text-lg font-semibold text-slate-800">
                {form.id ? "Edit student" : "Add student"}
            </h3>
            <form onSubmit={handleSubmit} className="grid gap-3 md:grid-cols-2">
                <div>
                    <label
                        htmlFor="student-unique-id"
                        className="mb-1 block text-sm font-medium text-slate-700"
                    >
                        Student Unique ID
                    </label>

```

```

        <input
          id="student-unique-id"
          value={form.studentUniqueId || "Generating..."}
          readOnly
          className="w-full rounded-lg border border-slate-200 bg-slate-100 px-3
py-2 text-slate-700 outline-none"
        />
      </div>
      <div>
        <label htmlFor="student-name" className="mb-1 block text-sm font-medium
text-slate-700">
          Name
        </label>
        <input
          id="student-name"
          required
          value={form.name}
          onChange={(event) => setForm((prev) => ({ ...prev, name:
event.target.value })))}
          className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
        />
      </div>
      <div>
        <label htmlFor="student-email" className="mb-1 block text-sm font-medium
text-slate-700">
          Email
        </label>
        <input
          id="student-email"
          type="email"
          required
          value={form.email}
          onChange={(event) => setForm((prev) => ({ ...prev, email:
event.target.value })))}
          className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
        />
      </div>
      <div>
        <label
          htmlFor="student-department"
          className="mb-1 block text-sm font-medium text-slate-700"
        >
          Department
        </label>
        <input
          id="student-department"
          required
          value={form.department}
          onChange={(event) =>
            setForm((prev) => ({ ...prev, department: event.target.value })))
        />
      </div>

```

```

    }
    className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
  />
</div>
<div>
  <label
    htmlFor="student-semester"
    className="mb-1 block text-sm font-medium text-slate-700"
  >
    Semester
  </label>
  <input
    id="student-semester"
    type="number"
    min="1"
    max="8"
    required
    value={form.semester}
    onChange={(event) =>
      setForm((prev) => ({ ...prev, semester: event.target.value }))
    }
    className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
  />
</div>
<div className="md:col-span-2">
  <button
    type="submit"
    disabled={loading}
    className="cursor-pointer rounded-lg bg-purple-600 px-4 py-2 text-sm
font-medium text-white transition-colors duration-200 hover:bg-purple-700
disabled:cursor-not-allowed disabled:bg-purple-300"
  >
    {loading ? "Saving..." : form.id ? "Update student" : "Create
student"}
  </button>
</div>
</form>
</section>

<section className="rounded-xl border border-slate-100 p-4">
  <h3 className="mb-4 text-lg font-semibold text-slate-800">Student
records</h3>
  <div className="space-y-2">
    {students.map((student) => (
      <article
        key={student._id}
        className="flex flex-wrap items-center justify-between gap-3 rounded-
lg border border-slate-100 bg-slate-50 px-3 py-2"
      >
        <p className="text-sm text-slate-700">

```

```

        <span className="font-medium">{student.name}</span> · ID:{" "}
        <span className="font-medium">{student.studentUniqueId || "N/A"}
    </span> · {" "}
        {student.email} · {" "}
        {student.department} · Sem {student.semester}
    </p>
    <div className="space-x-2">
        <button
            type="button"
            onClick={() =>
                setForm({
                    id: student._id,
                    studentUniqueId: student.studentUniqueId || "",
                    name: student.name,
                    email: student.email,
                    department: student.department,
                    semester: student.semester,
                })
            }
            className="cursor-pointer rounded bg-amber-500 px-3 py-1 text-xs
font-medium text-white transition-colors duration-200 hover:bg-amber-600"
        >
            Edit
        </button>
        <button
            type="button"
            onClick={() => handleDelete(student._id)}
            className="cursor-pointer rounded bg-rose-600 px-3 py-1 text-xs
font-medium text-white transition-colors duration-200 hover:bg-rose-700"
        >
            Delete
        </button>
    </div>
</article>
    )}}
</div>
</section>
</div>
);
}

export default StudentsPage;

```

frontend/src/pages/CoursesPage.jsx

Work of this code: Manages course catalog with create/update/delete operations and list rendering.

```

import { useEffect, useState } from "react";
import { apiRequest } from "../api";

const initialForm = { id: "", code: "", title: "", department: "", credits: 1 };

```

```

function CoursesPage() {
  const [courses, setCourses] = useState([]);
  const [form, setForm] = useState(initialForm);
  const [status, setStatus] = useState("");
  const [loading, setLoading] = useState(false);

  async function loadCourses() {
    const data = await apiRequest("/courses");
    setCourses(data);
  }

  useEffect(() => {
    let ignore = false;

    async function bootstrap() {
      try {
        const data = await apiRequest("/courses");
        if (!ignore) {
          setCourses(data);
        }
      } catch (error) {
        if (!ignore) {
          setStatus(error.message);
        }
      }
    }

    bootstrap();

    return () => {
      ignore = true;
    };
  }, []);

  async function handleSubmit(event) {
    event.preventDefault();
    setLoading(true);
    setStatus("");
    try {
      const payload = {
        code: form.code,
        title: form.title,
        department: form.department,
        credits: Number(form.credits),
      };
      if (form.id) {
        await apiRequest(`/courses/${form.id}`, {
          method: "PUT",
          body: JSON.stringify(payload),
        });
        setStatus("Course updated successfully.");
      }
    }
  }
}

```

```

    } else {
      await apiRequest("/courses", { method: "POST", body: JSON.stringify(payload)
    });
    setStatus("Course created successfully.");
  }
  setForm(initialForm);
  await loadCourses();
} catch (submitError) {
  setStatus(submitError.message);
} finally {
  setLoading(false);
}
}

async function handleDelete(id) {
  try {
    await apiRequest(`/courses/${id}`, { method: "DELETE" });
    setStatus("Course deleted successfully.");
    await loadCourses();
  } catch (deleteError) {
    setStatus(deleteError.message);
  }
}

return (
  <div className="space-y-6">
    {status ? <p className="rounded bg-slate-100 px-3 py-2 text-sm text-slate-700">{status}</p> : null}

    <section className="rounded-xl border border-slate-100 p-4">
      <h3 className="mb-4 text-lg font-semibold text-slate-800">
        {form.id ? "Edit course" : "Add course"}
      </h3>
      <form onSubmit={handleSubmit} className="grid gap-3 md:grid-cols-2">
        <div>
          <label htmlFor="course-code" className="mb-1 block text-sm font-medium text-slate-700">
            Course code
          </label>
          <input
            id="course-code"
            required
            value={form.code}
            onChange={(event) => setForm((prev) => ({ ...prev, code:
event.target.value })))}
            className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
          />
        </div>
        <div>
          <label htmlFor="course-title" className="mb-1 block text-sm font-medium text-slate-700">

```

```

        Title
      </label>
      <input
        id="course-title"
        required
        value={form.title}
        onChange={(event) => setForm((prev) => ({ ...prev, title:
event.target.value })))}
        className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
      />
    </div>
    <div>
      <label
        htmlFor="course-department"
        className="mb-1 block text-sm font-medium text-slate-700"
      >
        Department
      </label>
      <input
        id="course-department"
        required
        value={form.department}
        onChange={(event) =>
          setForm((prev) => ({ ...prev, department: event.target.value }))
        }
        className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
      />
    </div>
    <div>
      <label
        htmlFor="course-credits"
        className="mb-1 block text-sm font-medium text-slate-700"
      >
        Credits
      </label>
      <input
        id="course-credits"
        type="number"
        min="1"
        max="10"
        required
        value={form.credits}
        onChange={(event) => setForm((prev) => ({ ...prev, credits:
event.target.value })))}
        className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
      />
    </div>
    <div className="md:col-span-2">
      <button

```

```

        type="submit"
        disabled={loading}
        className="cursor-pointer rounded-lg bg-purple-600 px-4 py-2 text-sm
font-medium text-white transition-colors duration-200 hover:bg-purple-700
disabled:cursor-not-allowed disabled:bg-purple-300"
      >
        {loading ? "Saving..." : form.id ? "Update course" : "Create course"}
      </button>
    </div>
  </form>
</section>

<section className="rounded-xl border border-slate-100 p-4">
  <h3 className="mb-4 text-lg font-semibold text-slate-800">Course
catalog</h3>
  <div className="space-y-2">
    {courses.map((course) => (
      <article
        key={course._id}
        className="flex flex-wrap items-center justify-between gap-3 rounded-
lg border border-slate-100 bg-slate-50 px-3 py-2"
      >
        <p className="text-sm text-slate-700">
          <span className="font-medium">{course.code}</span> · {course.title}
        </p>
        {course.department} · {course.credits} credits
        </div>
        <div className="space-x-2">
          <button
            type="button"
            onClick={() =>
              setForm({
                id: course._id,
                code: course.code,
                title: course.title,
                department: course.department,
                credits: course.credits,
              })
            }
            className="cursor-pointer rounded bg-amber-500 px-3 py-1 text-xs
font-medium text-white transition-colors duration-200 hover:bg-amber-600"
          >
            Edit
          </button>
          <button
            type="button"
            onClick={() => handleDelete(course._id)}
            className="cursor-pointer rounded bg-rose-600 px-3 py-1 text-xs
font-medium text-white transition-colors duration-200 hover:bg-rose-700"
          >
            Delete
          </button>
        </div>
      </article>
    )}
  </div>

```

```

        </div>
      </article>
    )}
  </div>
</section>
</div>
);
}

export default CoursesPage;

```

frontend/src/pages/EnrollmentsPage.jsx

Work of this code: Creates student-course enrollments by semester and lists enrollment records with populated details.

```

import { useEffect, useMemo, useState } from "react";
import { apiRequest } from "../api";

function EnrollmentsPage() {
  const [students, setStudents] = useState([]);
  const [courses, setCourses] = useState([]);
  const [enrollments, setEnrollments] = useState([]);
  const [form, setForm] = useState({ studentId: "", courseId: "", semester: 1 });
  const [status, setStatus] = useState("");
  const [loading, setLoading] = useState(false);

  const canCreate = useMemo(
    () => students.length > 0 && courses.length > 0,
    [students.length, courses.length]
  );

  async function loadData() {
    const [studentsData, coursesData, enrollmentsData] = await Promise.all([
      apiRequest("/students"),
      apiRequest("/courses"),
      apiRequest("/enrollments"),
    ]);
    setStudents(studentsData);
    setCourses(coursesData);
    setEnrollments(enrollmentsData);
  }

  useEffect(() => {
    let ignore = false;

    async function bootstrap() {
      try {
        const [studentsData, coursesData, enrollmentsData] = await Promise.all([
          apiRequest("/students"),
          apiRequest("/courses"),

```

```

        apiRequest("/enrollments"),
    });
    if (!ignore) {
        setStudents(studentsData);
        setCourses(coursesData);
        setEnrollments(enrollmentsData);
    }
} catch (error) {
    if (!ignore) {
        setStatus(error.message);
    }
}
}

bootstrap();

return () => {
    ignore = true;
};
}, []);

async function handleSubmit(event) {
    event.preventDefault();
    setLoading(true);
    setStatus("");
    try {
        await apiRequest("/enrollments", {
            method: "POST",
            body: JSON.stringify({
                studentId: form.studentId,
                courseId: form.courseId,
                semester: Number(form.semester),
            }),
        });
        setStatus("Enrollment created successfully.");
        setForm({ studentId: "", courseId: "", semester: 1 });
        await loadData();
    } catch (submitError) {
        setStatus(submitError.message);
    } finally {
        setLoading(false);
    }
}

return (
    <div className="space-y-6">
        {status ? <p className="rounded bg-slate-100 px-3 py-2 text-sm text-slate-700">{status}</p> : null}

        <section className="rounded-xl border border-slate-100 p-4">
            <h3 className="mb-4 text-lg font-semibold text-slate-800">Create
enrollment</h3>

```

```

<form onSubmit={handleSubmit} className="grid gap-3 md:grid-cols-4">
  <div>
    <label
      htmlFor="enrollment-student"
      className="mb-1 block text-sm font-medium text-slate-700"
    >
      Student
    </label>
    <select
      id="enrollment-student"
      value={form.studentId}
      onChange={(event) =>
        setForm((prev) => ({ ...prev, studentId: event.target.value }))
      }
      className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
      required
    >
      <option value="">Select student</option>
      {students.map((student) => (
        <option key={student._id} value={student._id}>
          {student.name} ({student.studentUniqueId || "N/A"})
        </option>
      ))}
    </select>
  </div>
  <div>
    <label
      htmlFor="enrollment-course"
      className="mb-1 block text-sm font-medium text-slate-700"
    >
      Course
    </label>
    <select
      id="enrollment-course"
      value={form.courseId}
      onChange={(event) => setForm((prev) => ({ ...prev, courseId:
event.target.value })))}
      className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
      required
    >
      <option value="">Select course</option>
      {courses.map((course) => (
        <option key={course._id} value={course._id}>
          {course.code} - {course.title}
        </option>
      ))}
    </select>
  </div>
  <div>
    <label

```

```

        htmlFor="enrollment-semester"
        className="mb-1 block text-sm font-medium text-slate-700"
      >
        Semester
      </label>
      <input
        id="enrollment-semester"
        type="number"
        min="1"
        max="8"
        value={form.semester}
        onChange={(event) => setForm((prev) => ({ ...prev, semester:
event.target.value })))}
        className="w-full rounded-lg border border-slate-200 px-3 py-2 ring-
purple-300 outline-none focus:ring-2"
        required
      />
    </div>
    <div className="flex items-end">
      <button
        type="submit"
        disabled={!canCreate || loading}
        className="w-full cursor-pointer rounded-lg bg-purple-600 px-4 py-2
text-sm font-medium text-white transition-colors duration-200 hover:bg-purple-700
disabled:cursor-not-allowed disabled:bg-purple-300"
      >
        {loading ? "Saving..." : "Create"}
      </button>
    </div>
  </form>
</section>

<section className="rounded-xl border border-slate-100 p-4">
  <h3 className="mb-4 text-lg font-semibold text-slate-800">Enrollment
records</h3>
  <div className="space-y-2">
    {enrollments.map((enrollment) => (
      <article
        key={enrollment._id}
        className="rounded-lg border border-slate-100 bg-slate-50 px-3 py-2
text-sm text-slate-700"
      >
        <p className="mb-1 text-xs font-medium text-purple-700">
          Enrollment No: {enrollment.enrollmentNumber || "N/A"}
        </p>
        <p className="mb-1 text-xs font-medium text-slate-600">
          Student ID: {enrollment.student?.studentUniqueId || "N/A"}
        </p>
        <span className="font-medium">{enrollment.student?.name}</span>
enrolled in{" "}
        <span className="font-medium">{enrollment.course?.code}</span>
({enrollment.course?.title})
      </article>
    )
  )}
  </div>

```

```

        {" "}for semester {enrollment.semester}
      </article>
    )})
  </div>
</section>
</div>
);
}

export default EnrollmentsPage;

```

frontend/src/index.css

Work of this code: Loads Tailwind layers and applies base page styling for the app.

```

@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  @apply m-0 bg-slate-100 text-slate-900;
}

```

frontend/src/App.css

Work of this code: Contains additional app-level CSS styles from template/UI layer.

```

.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,

```

```

.vite {
  inset-inline: 0;
  margin: 0 auto;
}

.base {
  width: 170px;
  position: relative;
  z-index: 0;
}

.framework,
.vite {
  position: absolute;
}

.framework {
  z-index: 1;
  top: 34px;
  height: 28px;
  transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
    scale(1.4);
}

.vite {
  z-index: 0;
  top: 107px;
  height: 26px;
  width: auto;
  transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
    scale(0.8);
}
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

```

```

& > div {
  flex: 1 1 0;
  padding: 32px;
  @media (max-width: 1024px) {
    padding: 24px 20px;
  }
}

.icon {
  margin-bottom: 16px;
  width: 22px;
  height: 22px;
}

@media (max-width: 1024px) {
  flex-direction: column;
  text-align: center;
}
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;
  }
}

```

```

    &:hover {
      box-shadow: var(--shadow);
    }
    .button-icon {
      height: 18px;
      width: 18px;
    }
  }
}

@media (max-width: 1024px) {
  margin-top: 20px;
  flex-wrap: wrap;
  justify-content: center;

  li {
    flex: 1 1 calc(50% - 8px);
  }

  a {
    width: 100%;
    justify-content: center;
    box-sizing: border-box;
  }
}

#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }

  &::after {

```

```
    right: 0;  
    border-right-color: var(--border);  
  }  
}
```